

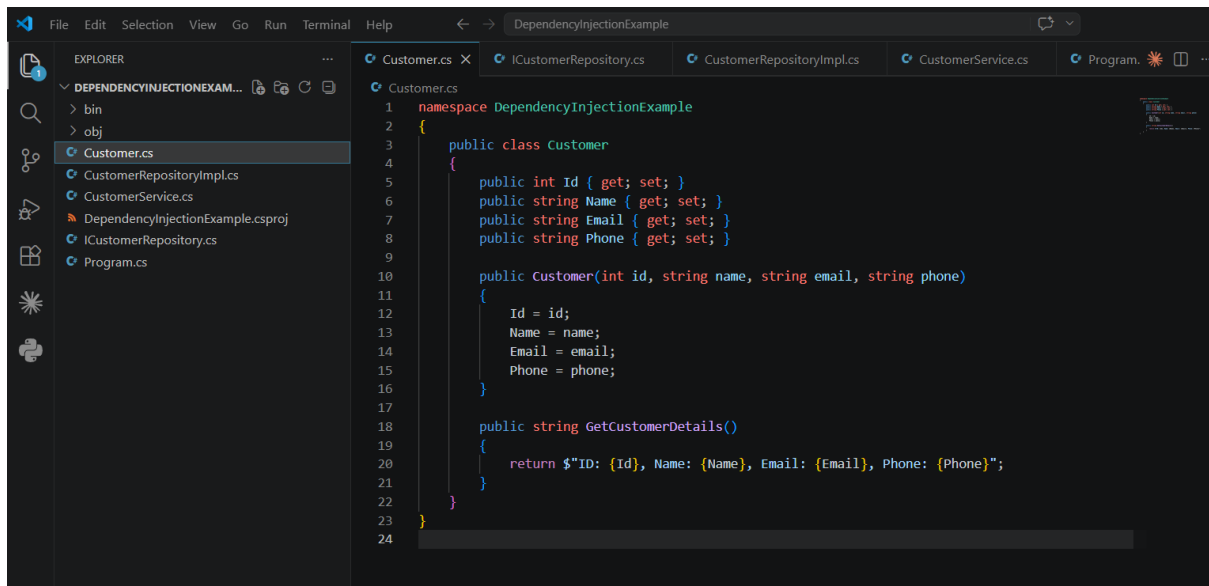
Hands-On Report

Exercise 11: Implementing Dependency Injection

In this exercise, I implemented and tested **Dependency Injection** in a .NET application by creating a project named **DependencyInjectionExample**. I developed a **CustomerRepository** interface and a **CustomerRepositoryImpl** class to manage customer data. A **CustomerService** class was created to depend on the repository interface rather than a specific implementation, and constructor-based dependency injection was used to provide the required dependency.

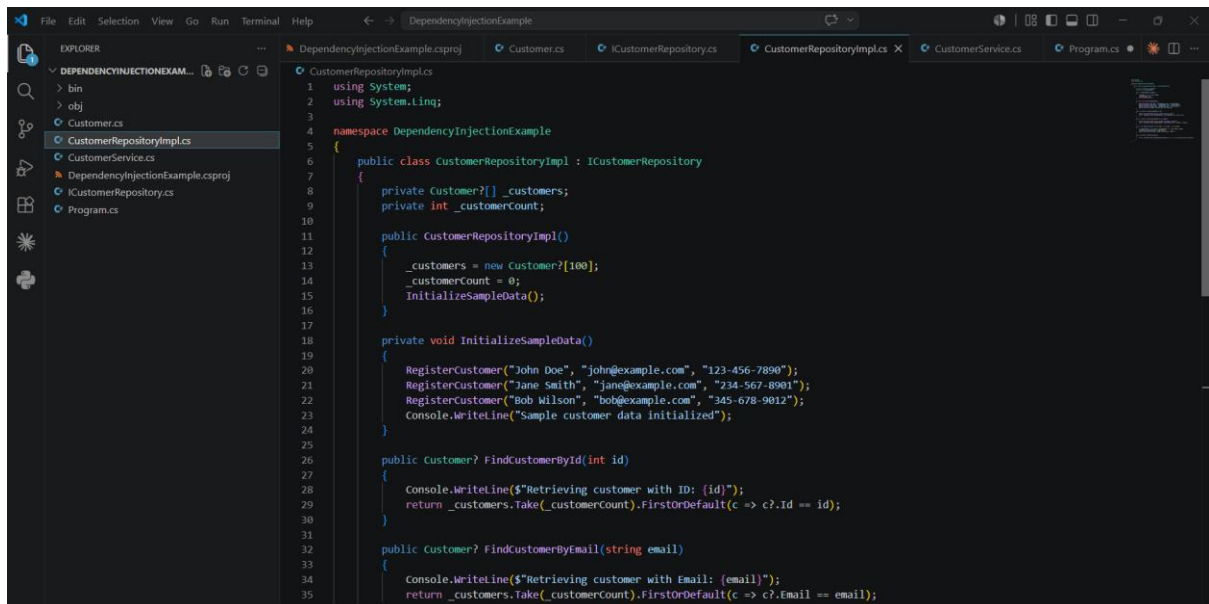
After executing the application, I verified that the service successfully retrieved customer information through the injected repository. This practical exercise helped me understand how Dependency Injection reduces tight coupling between components, improves flexibility, enhances code maintainability, and makes .NET applications easier to test by following the Dependency Inversion Principle.

Coding:



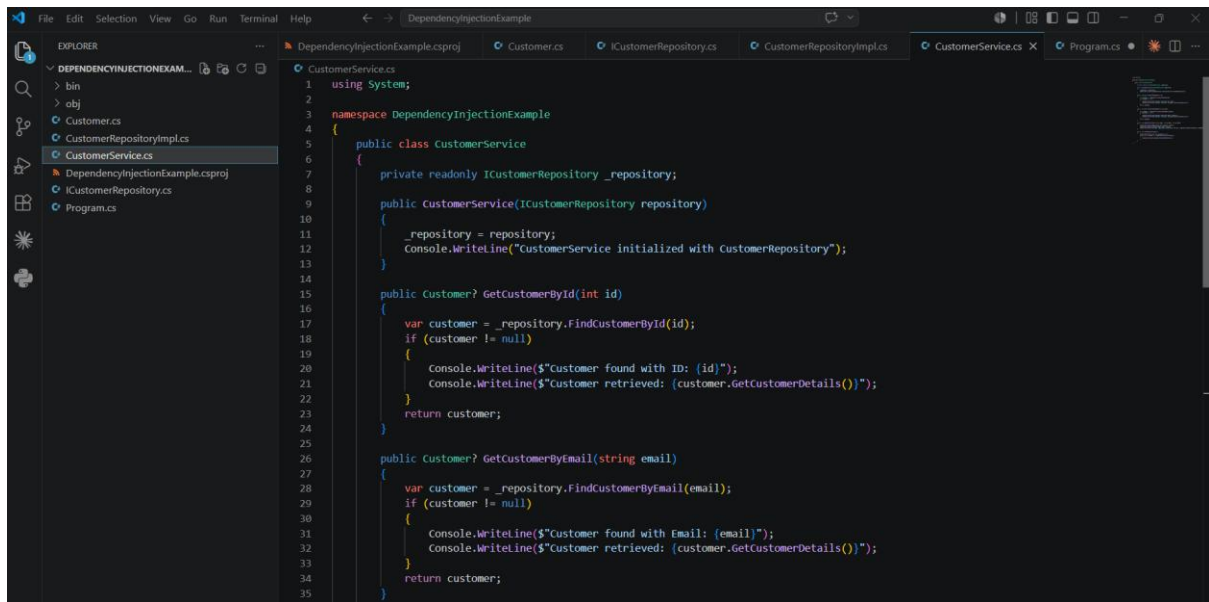
This screenshot shows the Visual Studio IDE with the 'DependencyInjectionExample' project open. The 'EXPLORER' sidebar on the left displays the project structure, including folders 'bin' and 'obj', and files 'Customer.cs', 'CustomerRepositoryImpl.cs', 'CustomerService.cs', 'DependencyInjectionExample.csproj', 'ICustomerRepository.cs', and 'Program.cs'. The 'Customer.cs' file is selected and its content is displayed in the main editor area. The code defines a 'Customer' class within the 'DependencyInjectionExample' namespace, featuring properties for Id, Name, Email, and Phone, a constructor, and a 'GetCustomerDetails()' method.

```
1 namespace DependencyInjectionExample
2 {
3     public class Customer
4     {
5         public int Id { get; set; }
6         public string Name { get; set; }
7         public string Email { get; set; }
8         public string Phone { get; set; }
9
10        public Customer(int id, string name, string email, string phone)
11        {
12            Id = id;
13            Name = name;
14            Email = email;
15            Phone = phone;
16        }
17
18        public string GetCustomerDetails()
19        {
20            return $"ID: {Id}, Name: {Name}, Email: {Email}, Phone: {Phone}";
21        }
22    }
23 }
24
```



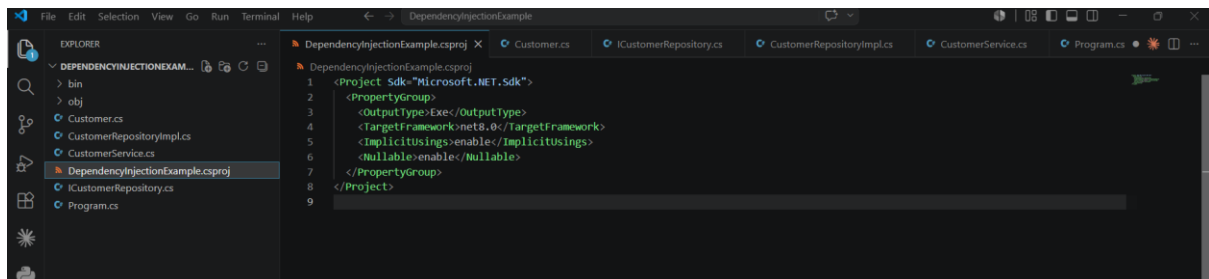
This screenshot shows the Visual Studio IDE with the 'DependencyInjectionExample' project open. The 'EXPLORER' sidebar on the left displays the project structure, with 'CustomerRepositoryImpl.cs' selected. The main editor area shows the implementation of the 'ICustomerRepository' interface. The 'CustomerRepositoryImpl' class includes a collection of customers, a count, and methods for initializing sample data and finding customers by ID or email.

```
1 using System;
2 using System.Linq;
3
4 namespace DependencyInjectionExample
5 {
6     public class CustomerRepositoryImpl : ICustomerRepository
7     {
8         private Customer?[] _customers;
9         private int _customerCount;
10
11        public CustomerRepositoryImpl()
12        {
13            _customers = new Customer?[100];
14            _customerCount = 0;
15            InitializeSampleData();
16        }
17
18        private void InitializeSampleData()
19        {
20            RegisterCustomer("John Doe", "john@example.com", "123-456-7890");
21            RegisterCustomer("Jane Smith", "jane@example.com", "234-567-8901");
22            RegisterCustomer("Bob Wilson", "bob@example.com", "345-678-9012");
23            Console.WriteLine("Sample customer data initialized");
24        }
25
26        public Customer? FindCustomerById(int id)
27        {
28            Console.WriteLine($"Retrieving customer with ID: {id}");
29            return _customers.Take(_customerCount).FirstOrDefault(c => c?.Id == id);
30        }
31
32        public Customer? FindCustomerByEmail(string email)
33        {
34            Console.WriteLine($"Retrieving customer with Email: {email}");
35            return _customers.Take(_customerCount).FirstOrDefault(c => c?.Email == email);
36        }
37    }
38 }
```



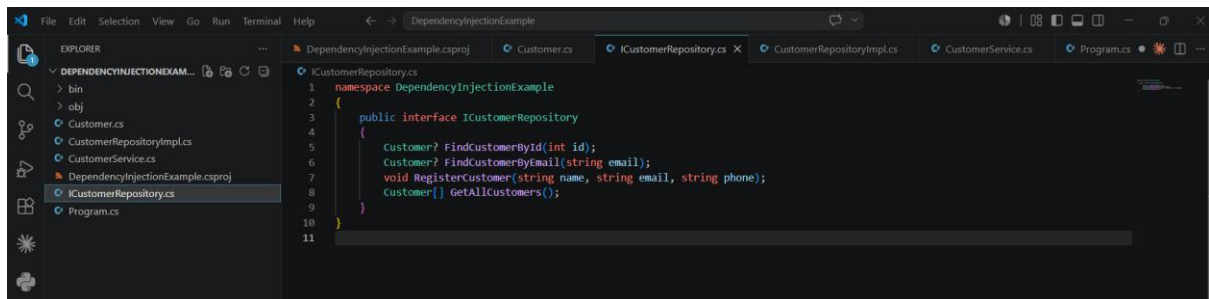
This screenshot shows the Visual Studio IDE with the `CustomerService.cs` file open. The Explorer pane on the left shows the project structure: `DependencyInjectionExample` (containing `bin`, `obj`, `Customer.cs`, `CustomerRepositoryImpl.cs`, `CustomerService.cs`, `DependencyInjectionExample.csproj`, `ICustomerRepository.cs`, and `Program.cs`). The main editor displays the following C# code:

```
1 using System;
2
3 namespace DependencyInjectionExample
4 {
5     public class CustomerService
6     {
7         private readonly ICustomerRepository _repository;
8
9         public CustomerService(ICustomerRepository repository)
10        {
11            _repository = repository;
12            Console.WriteLine("CustomerService initialized with CustomerRepository");
13        }
14
15        public Customer? GetCustomerById(int id)
16        {
17            var customer = _repository.FindCustomerById(id);
18            if (customer != null)
19            {
20                Console.WriteLine($"Customer found with ID: {id}");
21                Console.WriteLine($"Customer retrieved: {customer.GetCustomerDetails()}");
22            }
23            return customer;
24        }
25
26        public Customer? GetCustomerByEmail(string email)
27        {
28            var customer = _repository.FindCustomerByEmail(email);
29            if (customer != null)
30            {
31                Console.WriteLine($"Customer found with Email: {email}");
32                Console.WriteLine($"Customer retrieved: {customer.GetCustomerDetails()}");
33            }
34            return customer;
35        }
36    }
```



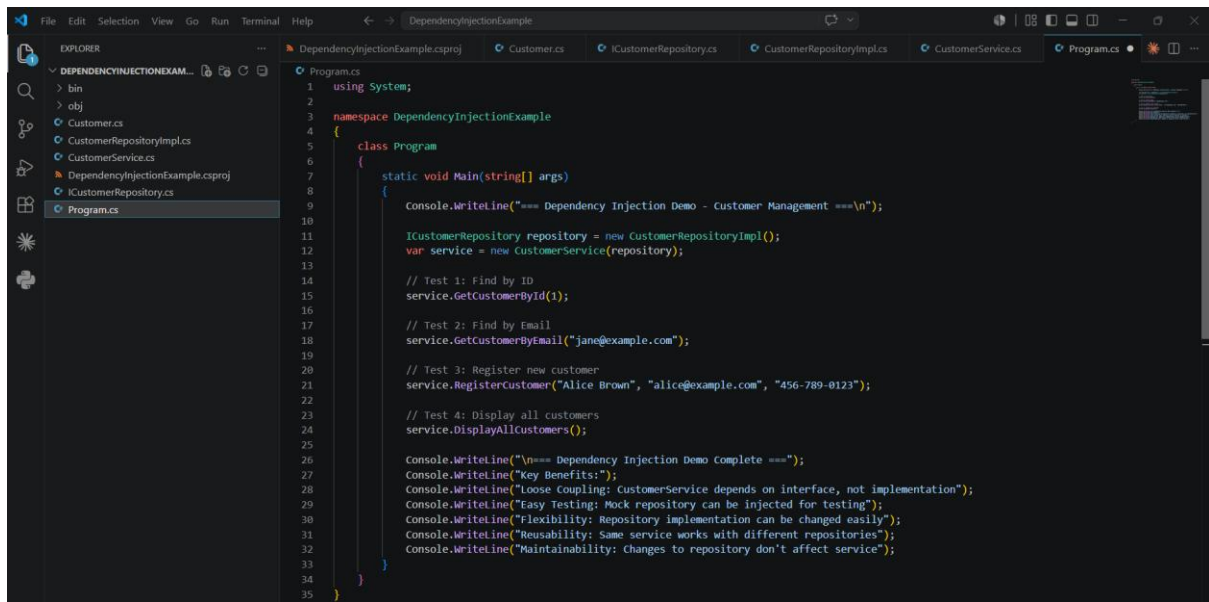
This screenshot shows the Visual Studio IDE with the `DependencyInjectionExample.csproj` file open. The Explorer pane on the left shows the project structure. The main editor displays the following XML code:

```
1 <Project Sdk="Microsoft.NET.Sdk">
2
3   <PropertyGroup>
4     <OutputType>Exe</OutputType>
5     <TargetFramework>net8.0</TargetFramework>
6     <ImplicitUsings>enable</ImplicitUsings>
7     <Nullable>enable</Nullable>
8   </PropertyGroup>
9 </Project>
```



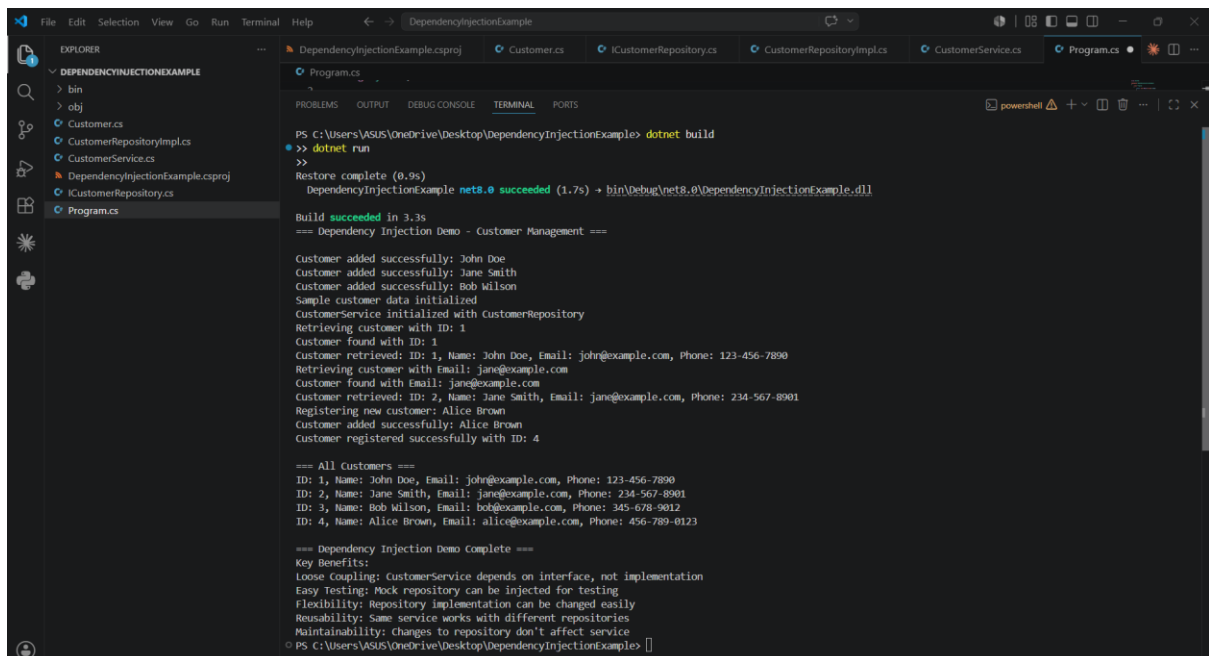
This screenshot shows the Visual Studio IDE with the `ICustomerRepository.cs` file open. The Explorer pane on the left shows the project structure. The main editor displays the following C# code:

```
1 namespace DependencyInjectionExample
2 {
3     public interface ICustomerRepository
4     {
5         Customer? FindCustomerById(int id);
6         Customer? FindCustomerByEmail(string email);
7         void RegisterCustomer(string name, string email, string phone);
8         Customer[] GetAllCustomers();
9     }
10 }
11
```



The screenshot shows the Visual Studio IDE with the 'EXPLORER' sidebar on the left displaying the project structure. The main editor window shows the code for `Program.cs` in the `DependencyInjectionExample` namespace. The code defines a `Program` class with a `Main` method that demonstrates dependency injection for a customer management system.

```
1 using System;
2
3 namespace DependencyInjectionExample
4 {
5     class Program
6     {
7         static void Main(string[] args)
8         {
9             Console.WriteLine("=== Dependency Injection Demo - Customer Management ===\n");
10
11             ICustomerRepository repository = new CustomerRepositoryImpl();
12             var service = new CustomerService(repository);
13
14             // Test 1: Find by ID
15             service.GetCustomerById(1);
16
17             // Test 2: Find by Email
18             service.GetCustomerByEmail("jane@example.com");
19
20             // Test 3: Register new customer
21             service.RegisterCustomer("Alice Brown", "alice@example.com", "456-789-0123");
22
23             // Test 4: Display all customers
24             service.DisplayAllCustomers();
25
26             Console.WriteLine("\n=== Dependency Injection Demo Complete ===");
27             Console.WriteLine("Key Benefits:");
28             Console.WriteLine("Loose Coupling: CustomerService depends on interface, not implementation");
29             Console.WriteLine("Easy Testing: Mock repository can be injected for testing");
30             Console.WriteLine("Flexibility: Repository implementation can be changed easily");
31             Console.WriteLine("Reusability: Same service works with different repositories");
32             Console.WriteLine("Maintainability: Changes to repository don't affect service");
33         }
34     }
35 }
```



The screenshot shows the Visual Studio IDE with the 'TERMINAL' window open at the bottom. The terminal displays the output of the application after running the command `dotnet run`. The output shows the successful execution of the program, including the registration of a new customer and the display of all customers.

```
PS C:\Users\ASUS\OneDrive\Desktop\DependencyInjectionExample> dotnet build
>> dotnet run
>>
Restore complete (0.9s)
DependencyInjectionExample net8.0 succeeded (1.7s) -> bin\Debug\net8.0\DependencyInjectionExample.dll

Build succeeded in 3.3s
=== Dependency Injection Demo - Customer Management ===

Customer added successfully: John Doe
Customer added successfully: Jane Smith
Customer added successfully: Bob Wilson
Sample customer data initialized
CustomerService initialized with CustomerRepository
Retrieving customer with ID: 1
Customer found with ID: 1
Customer retrieved: ID: 1, Name: John Doe, Email: john@example.com, Phone: 123-456-7890
Retrieving customer with Email: jane@example.com
Customer found with Email: jane@example.com
Customer retrieved: ID: 2, Name: Jane Smith, Email: jane@example.com, Phone: 234-567-8901
Registering new customer: Alice Brown
Customer added successfully: Alice Brown
Customer registered successfully with ID: 4

=== All Customers ===
ID: 1, Name: John Doe, Email: john@example.com, Phone: 123-456-7890
ID: 2, Name: Jane Smith, Email: jane@example.com, Phone: 234-567-8901
ID: 3, Name: Bob Wilson, Email: bob@example.com, Phone: 345-678-9012
ID: 4, Name: Alice Brown, Email: alice@example.com, Phone: 456-789-0123

=== Dependency Injection Demo Complete ===
Key Benefits:
Loose Coupling: CustomerService depends on interface, not implementation
Easy Testing: Mock repository can be injected for testing
Flexibility: Repository implementation can be changed easily
Reusability: Same service works with different repositories
Maintainability: Changes to repository don't affect service
PS C:\Users\ASUS\OneDrive\Desktop\DependencyInjectionExample>
```